

Technical guide

This guide explains how PrintXpress is built and how to set it up. It is written for a developer or a marker. It covers Task F of the assignment, the technical documentation.

Overview

PrintXpress is a native Android app written in Kotlin with Jetpack Compose for the UI and Room over SQLite for storage. It runs fully offline. There is no backend and no network calls.

Item	Value
Language	Kotlin
UI	Jetpack Compose, no XML layouts
Storage	Room over SQLite
State	ViewModel with Compose state
Package	com.hiranya.printxpress
minSdk	24
targetSdk	36
Build	Gradle with the version catalog in gradle/libs.versions.toml

Architecture

The app uses a simple layered design so each part has one job. The layers, from top to bottom, are the UI, the view models, the repositories, and the data layer.

1. UI (Compose screens) shows state and sends user actions to a view model. It never touches the database directly.
2. View models hold the screen state and call repositories. They expose state to the UI with Compose state holders.
3. Repositories wrap the DAOs and give the rest of the app clean methods such as `placeOrder` or `login`.
4. Data layer holds the Room entities, the DAOs, and the database class.

This flow is shown end to end in the sequence diagram at [diagrams/05-sequence.drawio](#), using the place order journey as the example.

Project structure

```
app/src/main/java/com/hiranya/printxpress/  
data/  
  entity/      Room @Entity classes (User, Address, Category, Product, Order, OrderItem, SavedDesign, Notification, Promot.  
  dao/         @Dao interfaces, one per area  
  repository/  repositories that wrap the DAOs  
  util/        HashUtil.kt (password hashing) and SessionManager.kt (in-memory login state)  
  PrintXpressDatabase.kt  the @Database class, plus the seed callback
```

```
ui/  
  theme/      Color.kt, Type.kt, Theme.kt  
  screens/    one Composable file per screen  
  components/ small reusable Composables  
  viewmodel/  one ViewModel per screen area  
  MainActivity.kt  
docs/         all documentation and the diagrams
```

Setup and build

1. Install Android Studio (a recent stable version) with the Android SDK for API 36.
2. Open the project folder in Android Studio and let Gradle sync.
3. Pick an emulator or a connected phone running Android 7.0 or later.
4. Press Run to build and install the app.

The first launch creates the database and seeds the starting catalog, so products appear straight away.

Dependencies

All required libraries are already in the project.

`gradle/libs.versions.toml` includes:

```
[versions]
room = "2.6.1"
ksp = "2.0.21-1.0.28"

[libraries]
androidx-room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
androidx-room-ktx = { group = "androidx.room", name = "room-ktx", version.ref = "room" }
androidx-room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room" }

[plugins]
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

The root `build.gradle.kts` has the KSP plugin:

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}
```

`app/build.gradle.kts` has the Room and ViewModel dependencies:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

dependencies {
    // Room for local storage
    implementation(libs.androidx.room.runtime)
    implementation(libs.androidx.room.ktx)
    ksp(libs.androidx.room.compiler)
    // ViewModel for Compose
    implementation(libs.androidx.lifecycle.viewmodel.compose)
}
```

Database

The schema, every table, and every column are in [04-database-design.md](#). The short version for setup follows.

1. Each table is a Room [@Entity](#). Foreign keys link children to parents, with cascade delete where a child cannot exist without its parent.
2. Each area has a [@Dao](#) interface with the queries it needs. DAO methods that touch the database are [suspend](#) functions so they run off the main thread.
3. [PrintXpressDatabase](#) is the [@Database](#) class that lists the entities and exposes the DAOs. It is built as a single instance for the app.
4. The starting catalog is seeded with a [RoomDatabase.Callback](#) the first time the database is created.

Make the database portable

To hand a filled database to a marker, export the SQLite file after running the app once.

1. In Android Studio, open View, then Tool Windows, then Device Explorer.

2. Go to `data/data/com.hiranya.printxpress/databases/`.
3. Right click `printxpress.db` and choose Save As to copy it to your computer.
4. Open the saved file in DB Browser for SQLite (free) to view the tables and rows.

If a pre-filled database is preferred over the seed callback, place the file at `app/src/main/assets/database/printxpress.db` and build the database with Room's `createFromAsset("database/printxpress.db")`. Use one approach only, not both.

State management

State stays simple. A view model holds screen state in Compose state holders such as `mutableStateOf` or a `StateFlow`, and the UI reads it. User actions call view model functions, which call repositories, which call the DAOs. There is no shared global state and no manual threading beyond the `suspend` DAO calls.

Validation and security

1. Input is validated before any save, following the rules in `03-requirements.md`. This covers required fields, the email and phone formats, the password length, and the order item checks.

- 2. Passwords are hashed before they are saved. Plain text passwords are never stored or logged.
- 3. No sensitive data is written to logs.

Theme and colours

The colours are defined once in `ui/theme/Color.kt` and used through the Material 3 colour scheme. The accent is the brand purple on a white background with dark gray text. Screens use the theme names, not raw hex values.

Role	Hex
Accent (primary)	#A20BC8
Accent container	#F6E7FA
On accent	#FFFFFF
Background and surface	#FFFFFF
Text primary	#1A1A1A
Text secondary	#5C5C5C
Divider	#E0E0E0

Map `primary` to the accent, `onPrimary` to white, `primaryContainer` to the accent container, `background` and `surface` to white, and `onBackground` and `onSurface` to text primary.

Testing

1. Unit tests run from Android Studio with Run, on the `test` source set.
2. Instrumented and UI tests run on an emulator or device, on the `androidTest` source set.
3. The full test plan, the test data, and the test cases are in `07-test-plan.md`.

Limitations

These are out of scope for this version, by design.

1. No online payment. Orders are paid on pickup or delivery.
2. No staff or admin app. Order status changes are simulated for the demo.

3. No real SMS or push delivery. Notifications are stored and shown inside the app.

4. No cloud sync. All data stays on the one device.